

CSCI 210: Computer Architecture

Lecture 12: Procedures & The Stack

Stephen Checkoway

Slides from Cynthia Taylor

Announcements

- Problem Set 4 Due Friday
- Lab 3 due Sunday

CS History: IBM System 360



- Family of mainframes developed in 1964
- Introduced:
 - 8-bit byte
 - Byte-addressable memory
 - 32-bit words
- Featured BAL (Branch and Link) and BR (Branch Register) instructions
- IBM's current System z mainframes will still run code written for the 360 series

From last time: Complete example

foo:

```
addi    sp, sp, -32      # Allocate space for stack frame
sd      ra, 24(sp)       # Stores (spills) ra, return address
sd      s0, 16(sp)       # Stores (spills) s0, callee-saved reg
...
li      s0, 25           # Set s0 to 25
sd      t3, 8(sp)        # Stores (spills) t3, caller-saved reg
add     a0, t1, t3
jal     ra, myFunction
ld      t3, 8(sp)        # Restores (fills) t3
...
ld      s0, 16(sp)       # Restores (fills) s0, must restore
ld      ra, 24(sp)       # Restores (fills) ra, return address
addi    sp, sp, 32       # Restore the stack pointer
jalr    zero, 0(ra)      # Return
```

From last time: Complete example

foo:

```
addi    sp, sp, -32
sd      ra, 24(sp)
sd      s0, 16(sp)
...
li      s0, 25
sd      t3, 8(sp)
add     a0, t1, t3
jal     ra, myFunction
ld      t3, 8(sp)
...
ld      s0, 16(sp)
ld      ra, 24(sp)
addi    sp, sp, 32
jalr    zero, 0(ra)
```

At the start of the function:

Arguments are in `a0, ..., a7` and then on the stack at `0(sp)`, `8(sp)`, ... at the start of the function

Stack frame for foo (32 bytes in size)

<code>sp + 24</code>	Saved return address <code>ra</code>
<code>sp + 16</code>	Saved register <code>s0</code>
<code>sp + 8</code>	Saved register <code>t3</code>
<code>sp + 0</code>	Unused space to preserve 16-byte alignment

Leaf function

- If the function doesn't call any other functions, it's a "leaf"
- If a leaf function doesn't need to use any of the callee-saved registers (e.g., s0–s11), then it doesn't need to change the stack pointer or spill/fill ra
- Example:

```
# myFunction(int a0, int a1, int a2)
```

```
myFunction:
```

```
    add    t0, a0, a2  
    sub    a0, t0, a1  
    jalr   zero, 0(ra)
```

Non-Leaf Procedures

- Procedures that call other procedures
- Caller needs to allocate a stack frame
- Caller needs to save on the stack:
 - Its return address
 - Any arguments and temporaries needed after the call
- Restore from the stack after the call

Non-Leaf Procedure Example

- C code:

```
int fact (int n) {  
    if (n < 2)  
        return 1;  
    else  
        return n * fact(n - 1);  
}
```

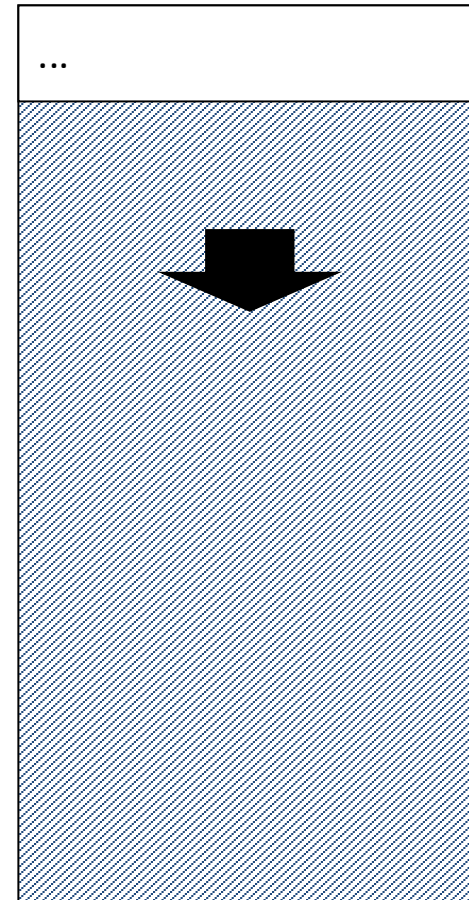
- Argument n in a0
- Result in a0

Process Stack

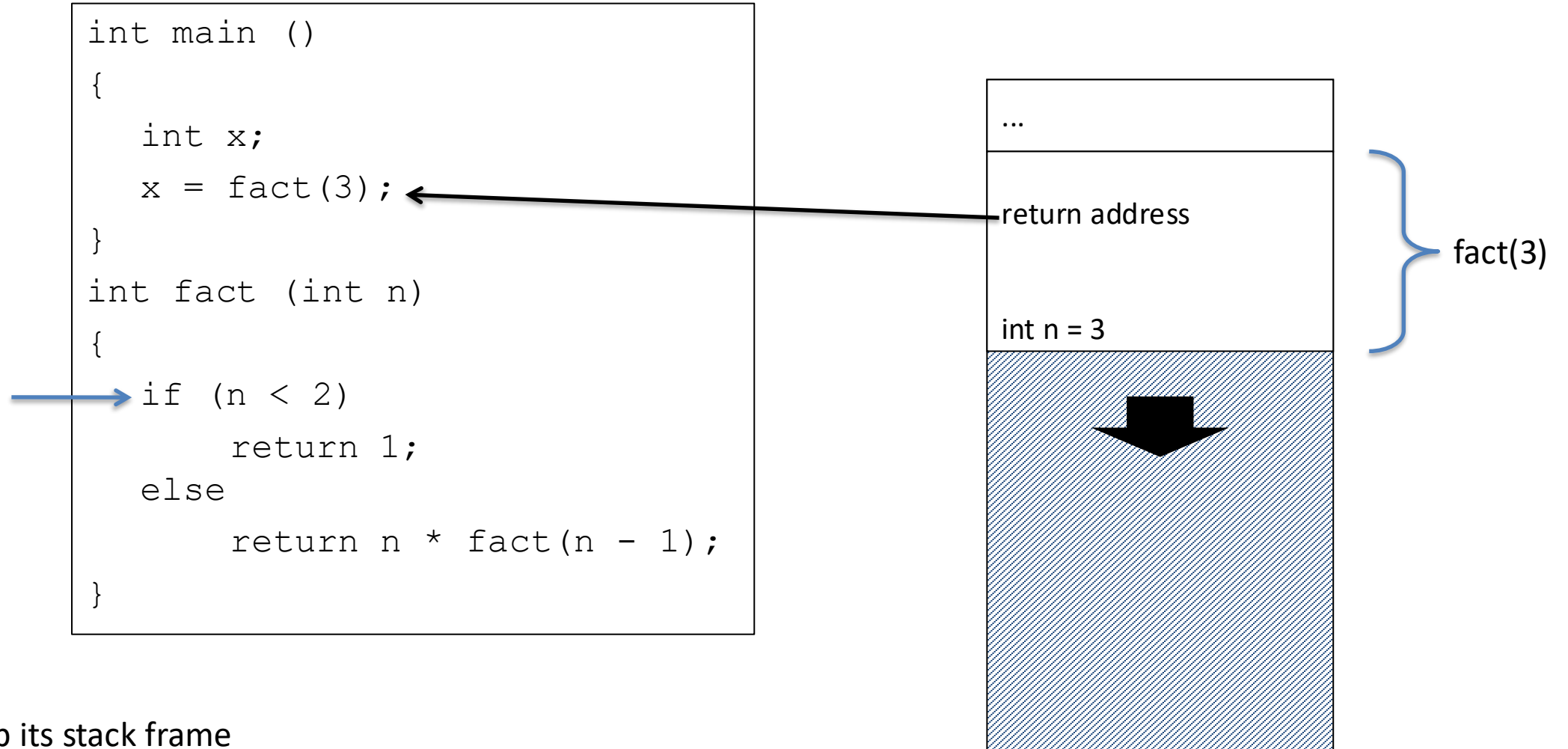
```
int main ()
{
    int x;
    → x = fact(3);
}
int fact (int n)
{
    if (n < 2)
        return 1;
    else
        return n * fact(n - 1);
}
```

Arrow is PC

We're about to call fact(3)



Process Stack

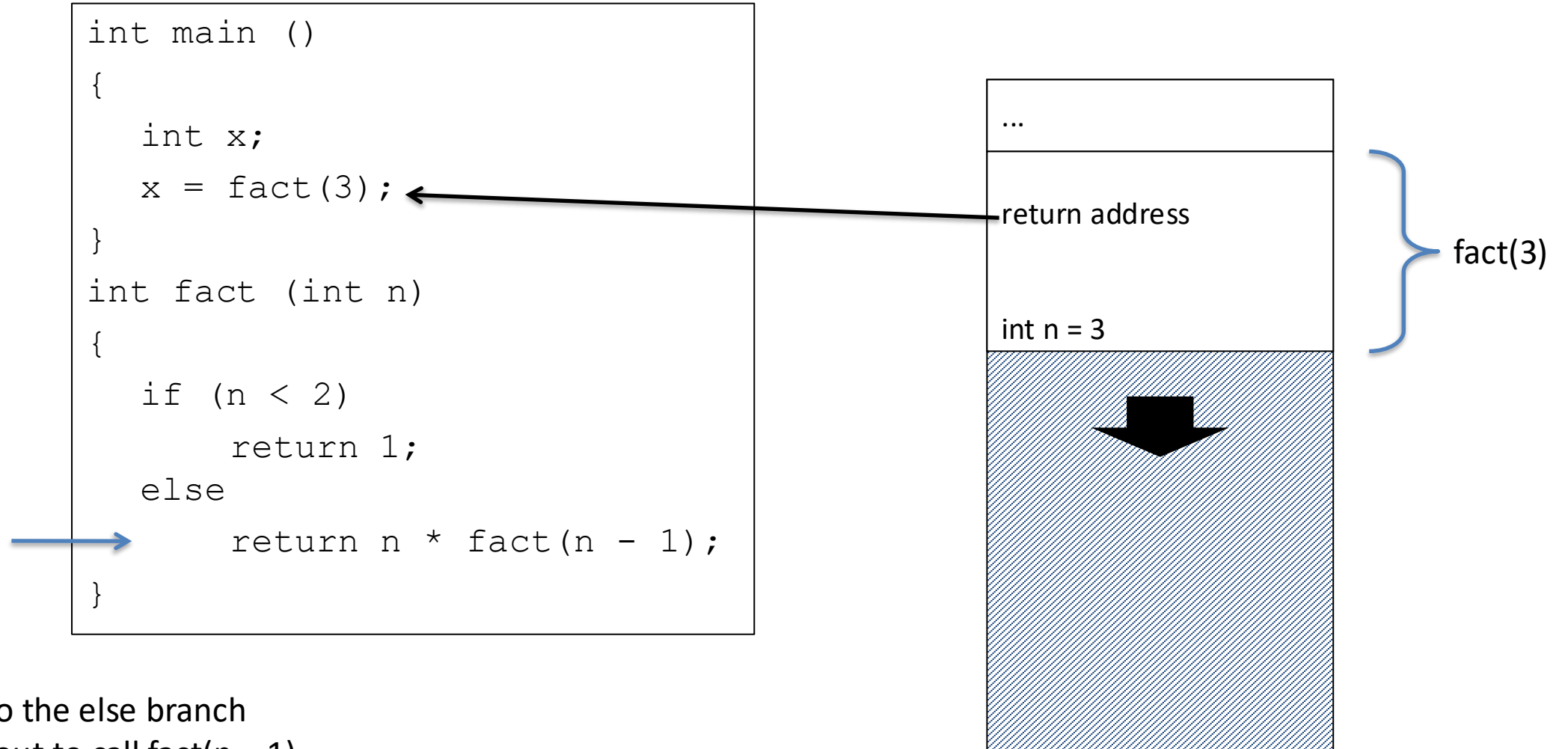


Fact sets up its stack frame

Return address points to just after `fact(3)` call

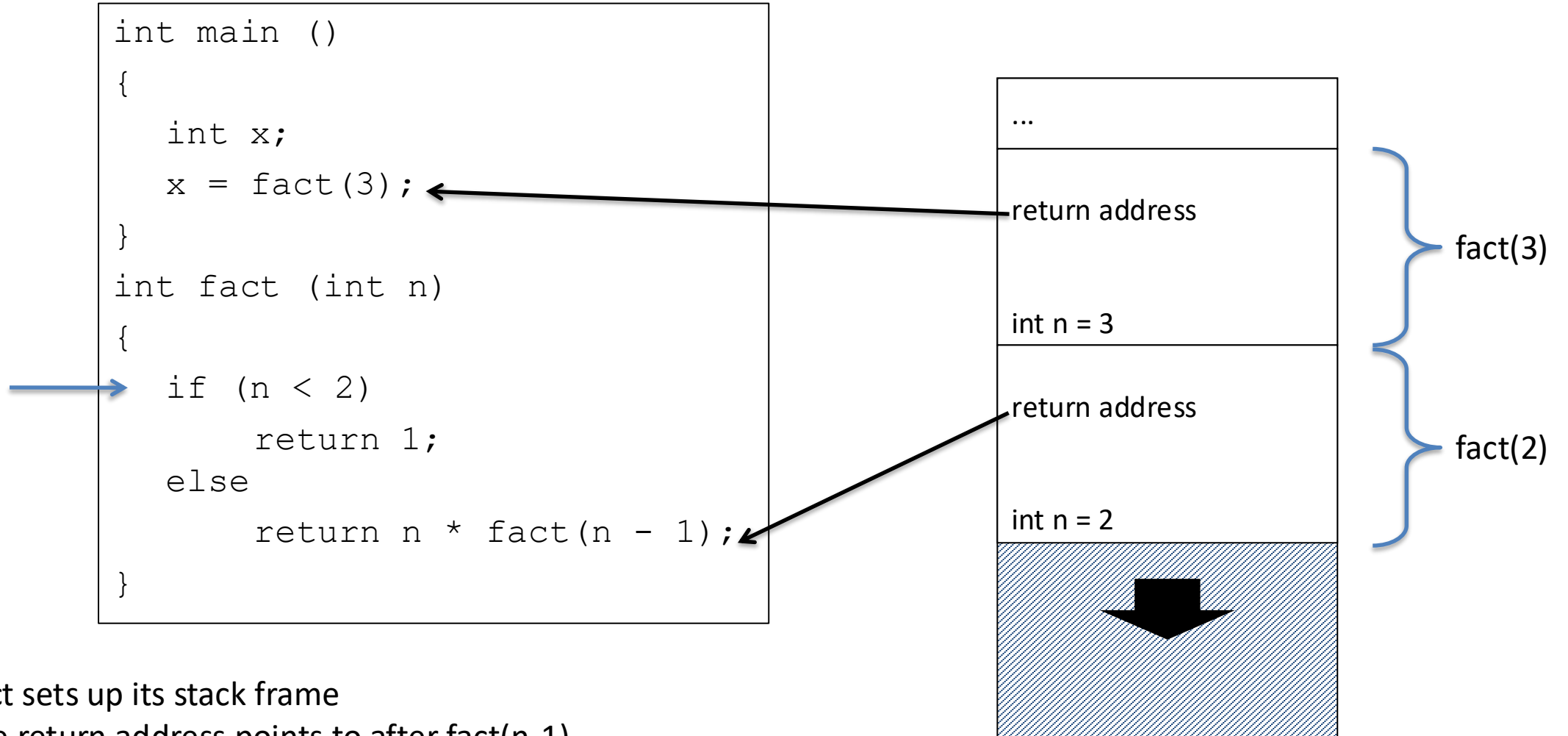
Since `n` is not less than 2...

Process Stack



...we go to the else branch
We're about to call `fact(n - 1)`

Process Stack

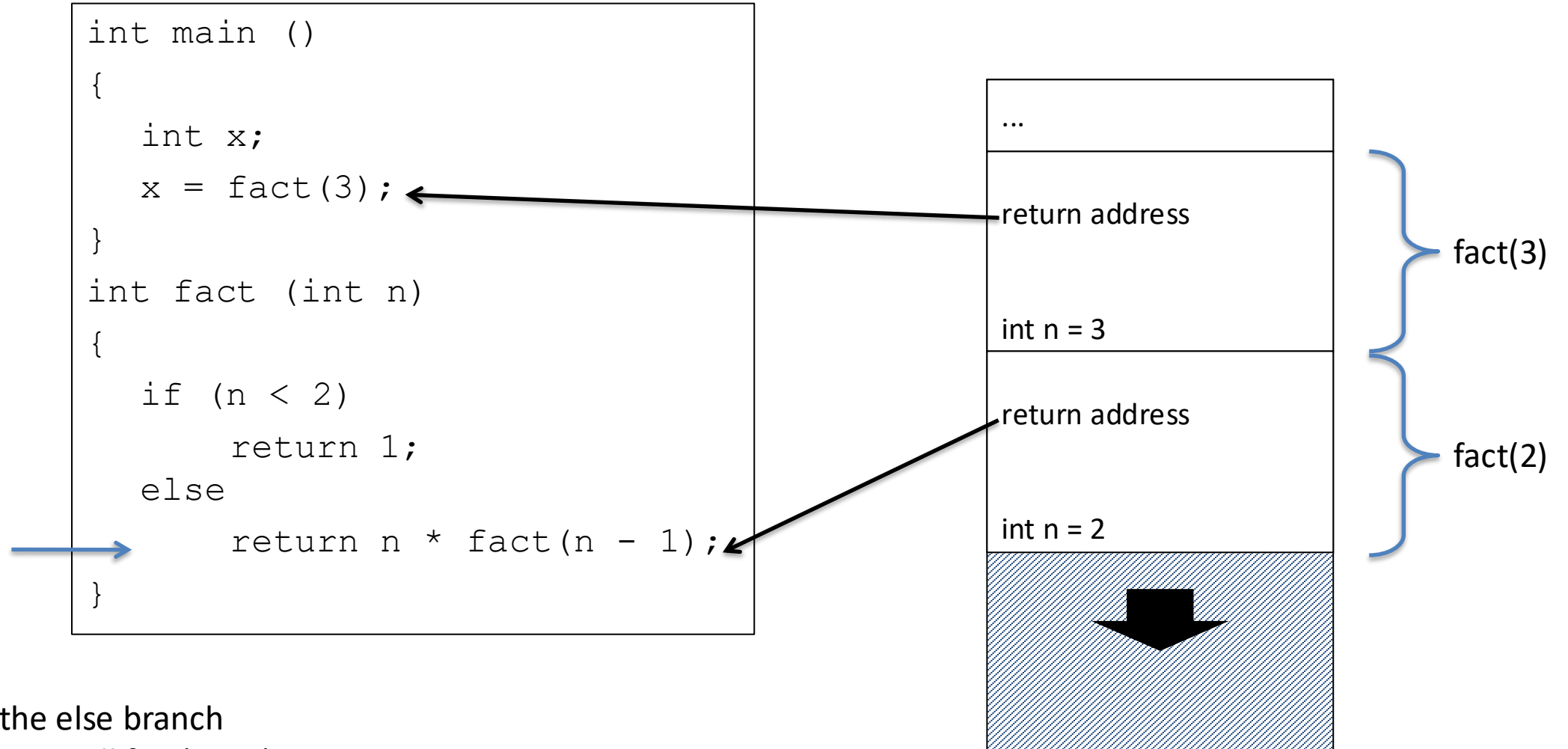


Fact sets up its stack frame

The return address points to after `fact(n-1)`

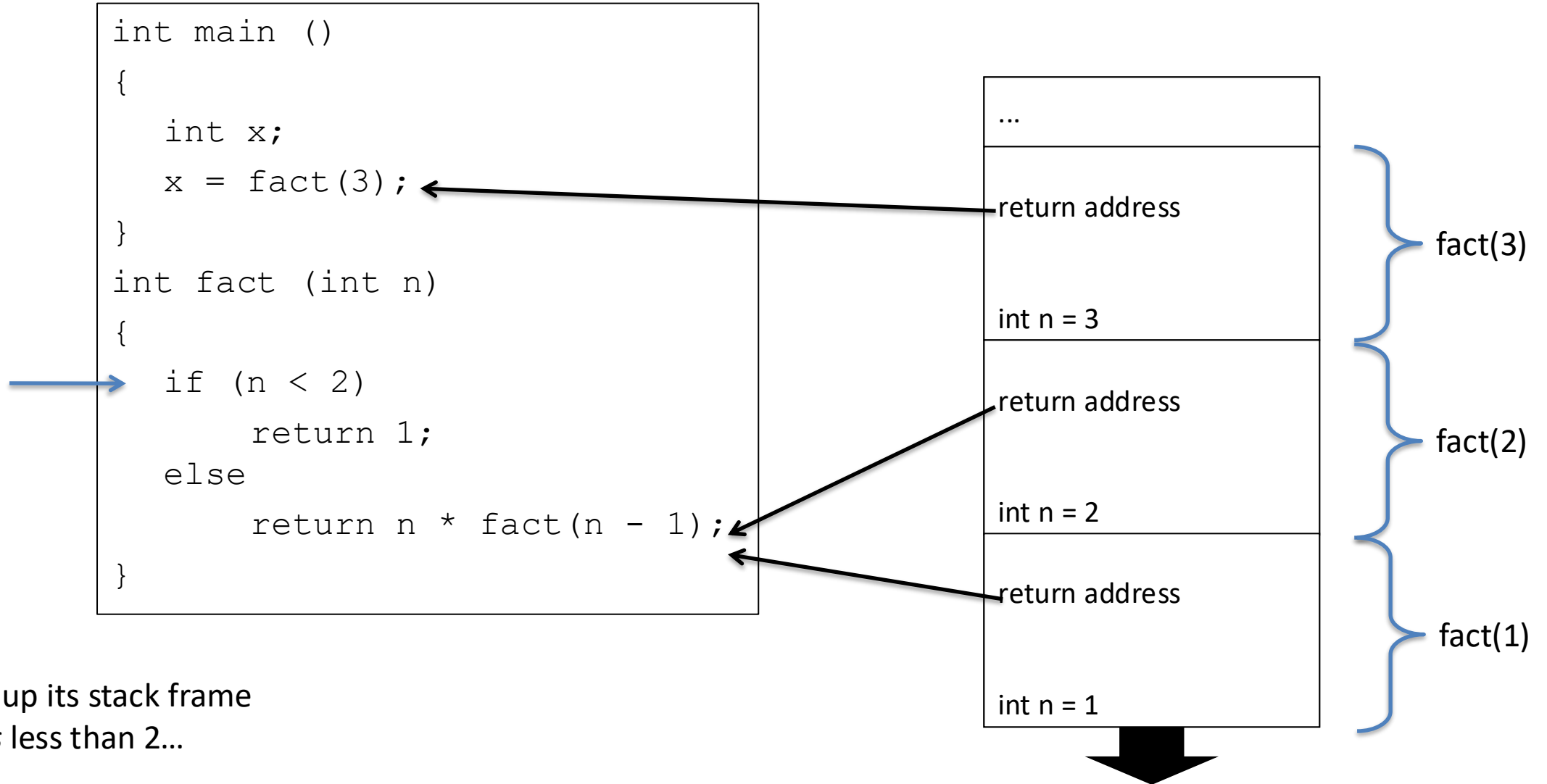
Since `n` is not less than 2...

Process Stack



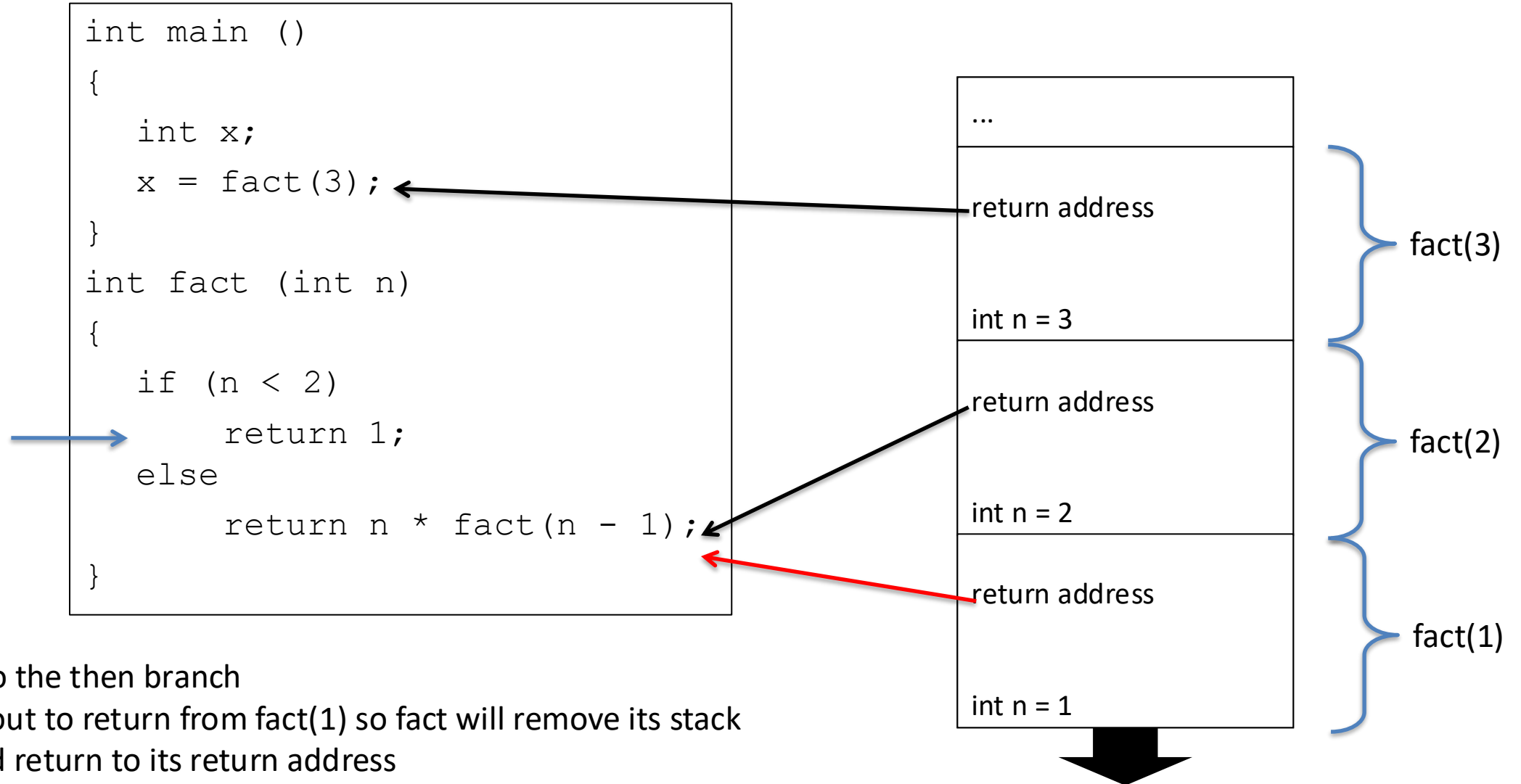
...we go to the else branch
We're about to call `fact(n - 1)`

Process Stack

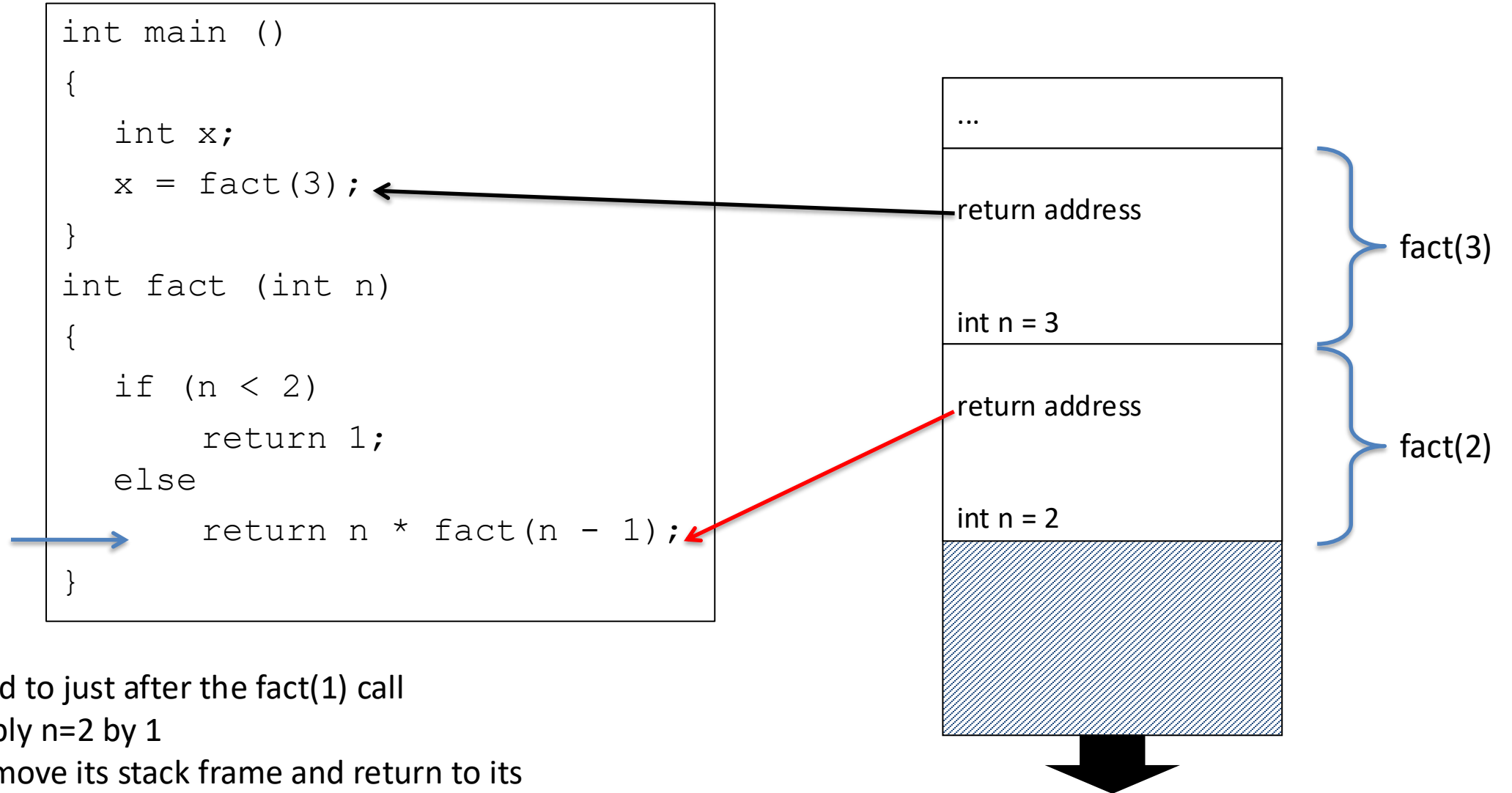


Fact sets up its stack frame
Since *n* is less than 2...

Process Stack

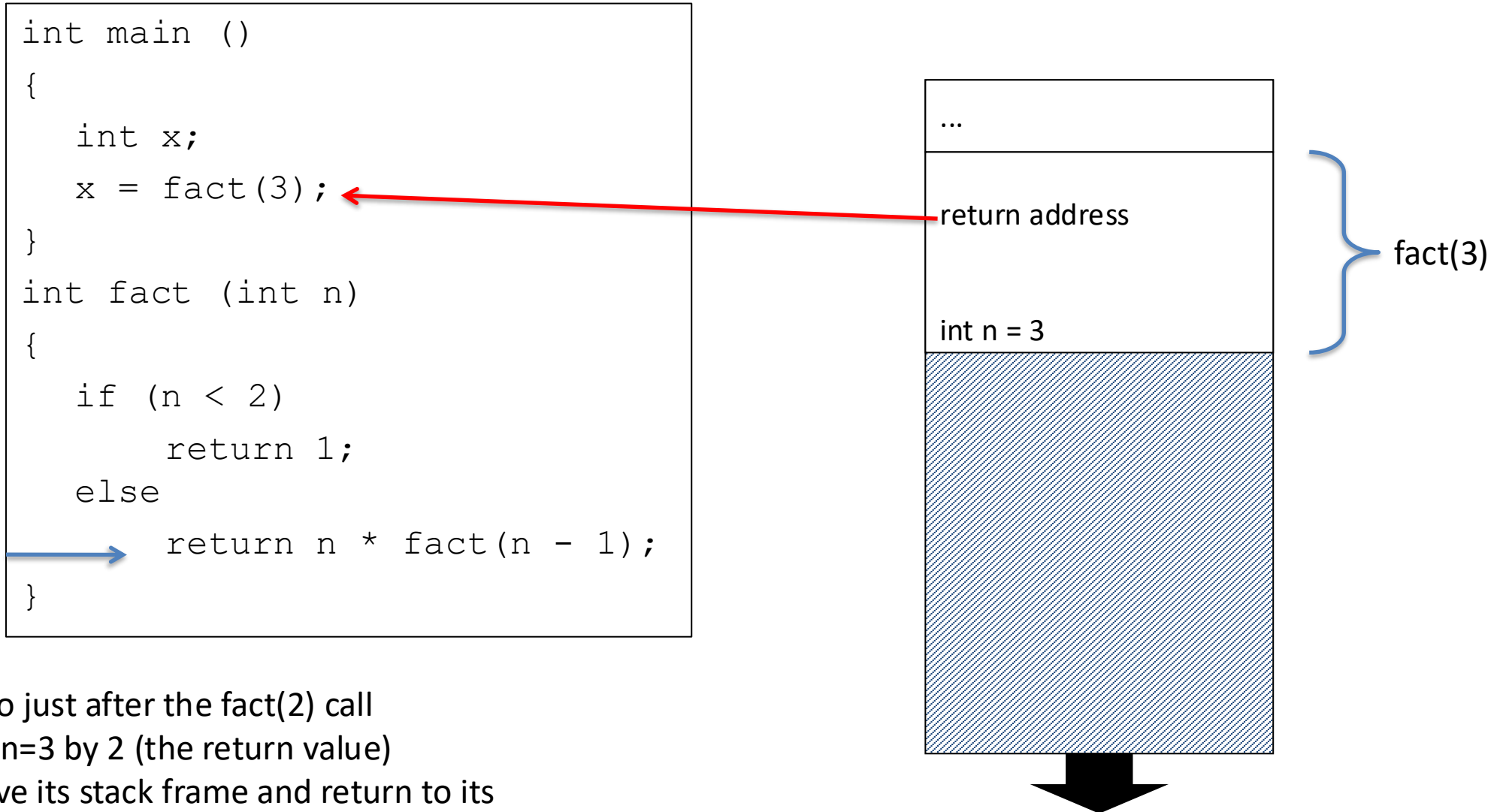


Process Stack



We returned to just after the `fact(1)` call
We'll multiply `n=2` by 1
Fact will remove its stack frame and return to its return address

Process Stack

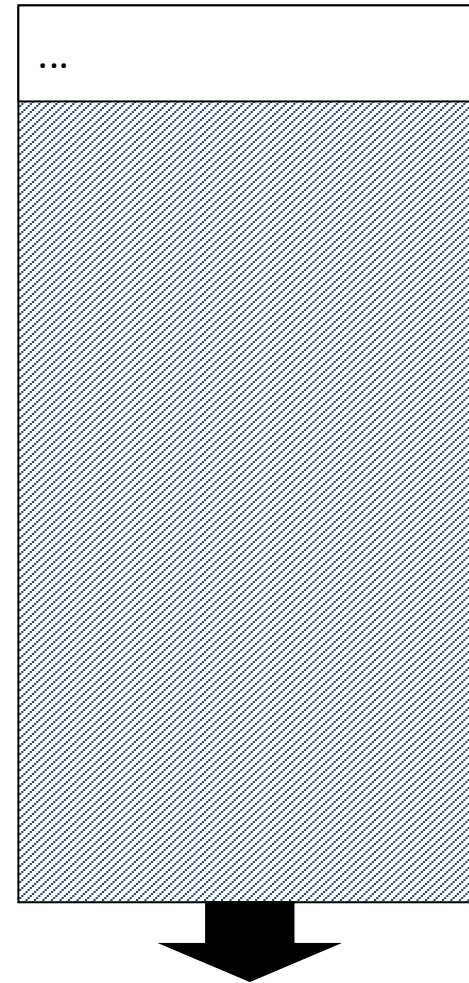


We returned to just after the `fact(2)` call
We'll multiply `n=3` by 2 (the return value)
Fact will remove its stack frame and return to its
return address

Process Stack

```
int main ()
{
    int x;
    x = fact(3);
}
int fact (int n)
{
    if (n < 2)
        return 1;
    else
        return n * fact(n - 1);
}
```

x now has the value 6



Questions?

Rules for allocating a stack frame for a nonleaf-procedure

Size of stack frame is sum of

- Local variables and temporaries
- $8 * \text{number of saved registers}$
- $8 * \text{number of arguments needed for called functions}$
beyond the 8 that fit in registers a0–a7 (this is very rare as most functions don't have more than 8 arguments!)

Round the whole thing up to a multiple of 16 for stack alignment

How many **local variables and arguments** does caller need to allocate space for on the stack?
Assume a 4-byte int (Hint: How many need to persist beyond a function call?)

- A. 1 (4 bytes)
- B. 2 (8 bytes)
- C. 3 (12 bytes)
- D. 4 (16 bytes)
- E. 5 (20 bytes)

```
int caller(int a, int b) {  
    int x = fun1();  
    int y = fun2(10, 3, a, b, 5);  
    int z = fun3(b, a);  
    return x + y + z;  
}
```

How many saved registers does caller need to allocate space for on the stack and which ones?

- A. 0
- B. 1 (pc)
- C. 1 (ra)
- D. 2 (pc, ra)
- E. 3 (ra, s0, s1)

```
int caller(int a, int b) {  
    int x = fun1();  
    int y = fun2(10, 3, a, b, 5);  
    int z = fun3(b, a);  
    return x + y + z;  
}
```

What is the total size of caller's stack frame if it needs to store four 4-byte integers (a, b, x, and y) and 1 saved register (ra)?

- A. 16 bytes
- B. 24 bytes
- C. 32 bytes
- D. 40 bytes
- E. 48 bytes

```
int caller(int a, int b) {  
    int x = fun1();  
    int y = fun2(10, 3, a, b, 5);  
    int z = fun3(b, a);  
    return x + y + z;  
}
```

Non-leaf recursive example

```
# fact(n)
```

```
fact:
```

```
    addi    sp, sp, -16      # allocate stack frame
    sd      ra, 8(sp)       # save return address
    sd      a0, 0(sp)       # save argument

    li      t0, 2           # if n < 2
    blt     a0, t0, L1      #   goto L1

    addi     a0, a0, -1      # else decrement n
    jal     ra, fact        # recursive call
    ld      t0, 0(sp)       # set t0 to the original n value
    mul     a0, a0, t0      # a0 = n * fact(n-1)
```

```
L1:
```

```
    ld      ra, 8(sp)       # restore ra
    addi     sp, sp, 16     # deallocate stack frame
    jalr     zero, 0(ra)    # return (return value is in a0)
```

At start of fact(3)

fact(n)

fact:

pc → **addi** sp, sp, -16

sd ra, 8(sp)

sd a0, 0(sp)

li t0, 2

blt a0, t0, L1

addi a0, a0, -1

jal ra, fact

ld t0, 0(sp)

mul a0, a0, t0

L1:

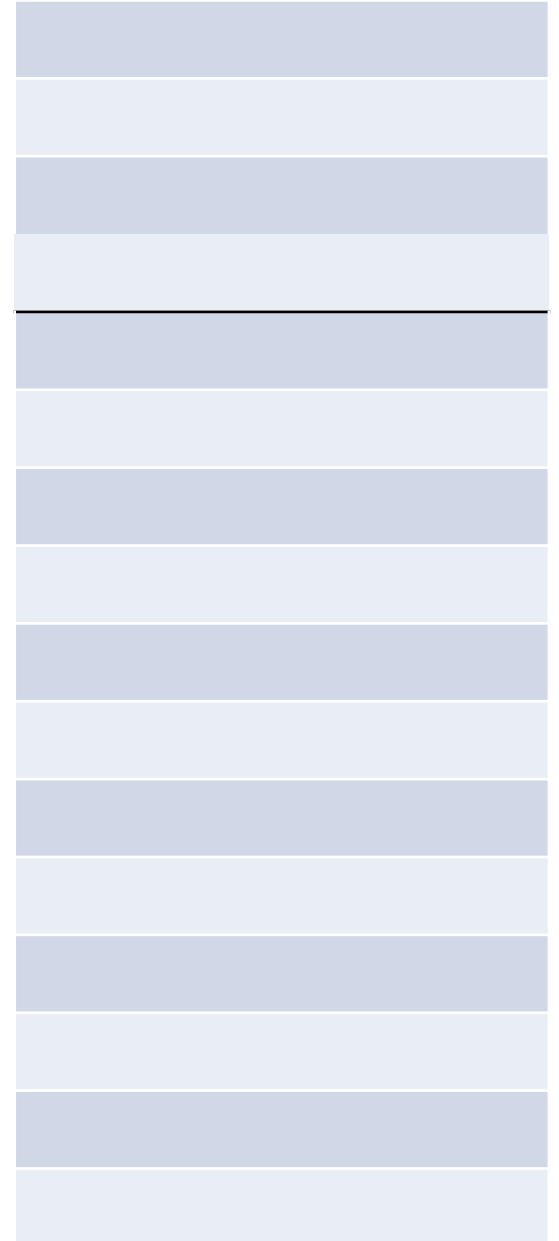
ld ra, 8(sp)

addi sp, sp, 16

jalr zero, 0(ra)

Reg	Value
a0	3
ra	main+20

sp →



After prologue

fact(n)
fact:

```
addi sp, sp, -16  
sd ra, 8(sp)  
sd a0, 0(sp)
```

pc →

```
li t0, 2  
blt a0, t0, L1
```

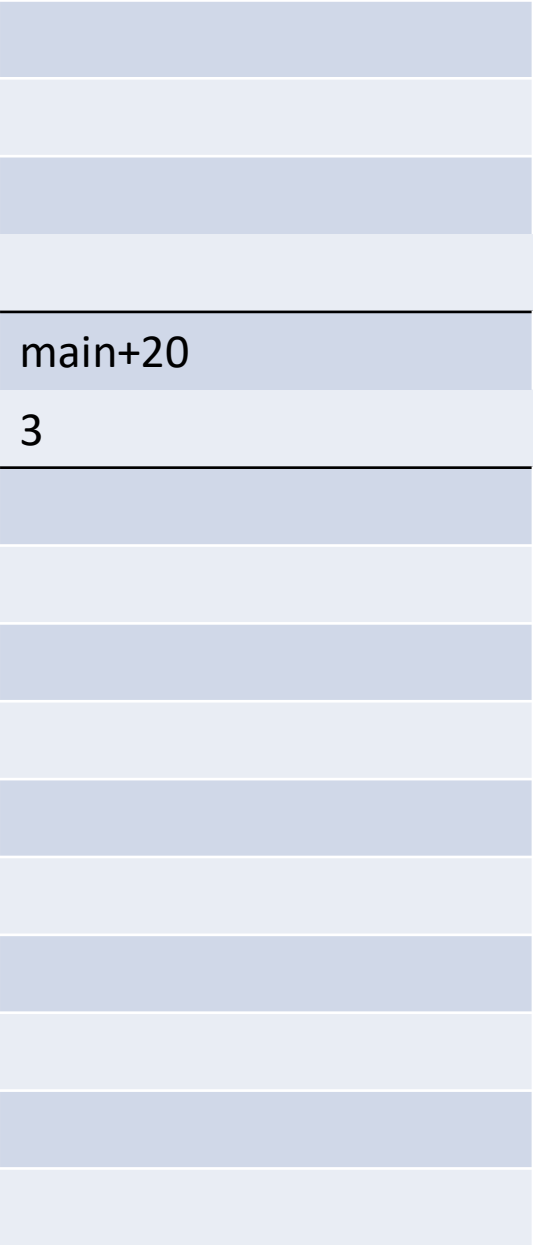
```
addi a0, a0, -1  
jal ra, fact  
ld t0, 0(sp)  
mul a0, a0, t0
```

L1:

```
ld ra, 8(sp)  
addi sp, sp, 16  
jalr zero, 0(ra)
```

Reg	Value
a0	3
ra	main+20

sp →



At start of fact(2)

fact(n)

fact:

pc → **addi** sp, sp, -16
sd ra, 8(sp)
sd a0, 0(sp)

li t0, 2
blt a0, t0, L1

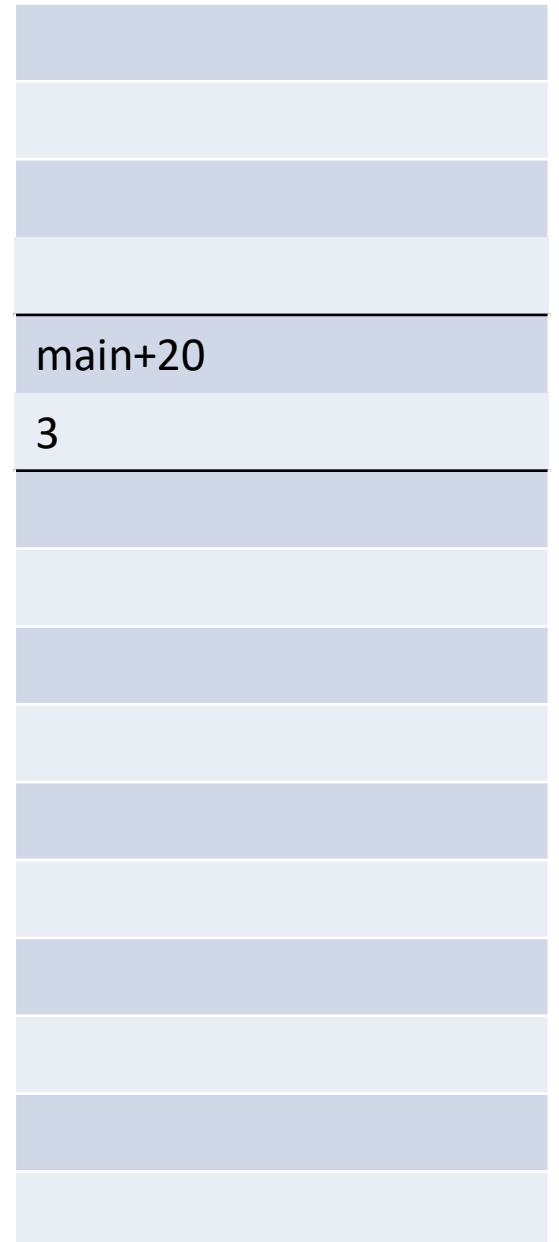
addi a0, a0, -1
jal ra, fact
ld t0, 0(sp)
mul a0, a0, t0

L1:

ld ra, 8(sp)
addi sp, sp, 16
jalr zero, 0(ra)

Reg	Value
a0	2
ra	fact+28

sp →



After prologue

```
# fact(n)
fact:
    addi    sp, sp, -16
    sd      ra, 8(sp)
    sd      a0, 0(sp)

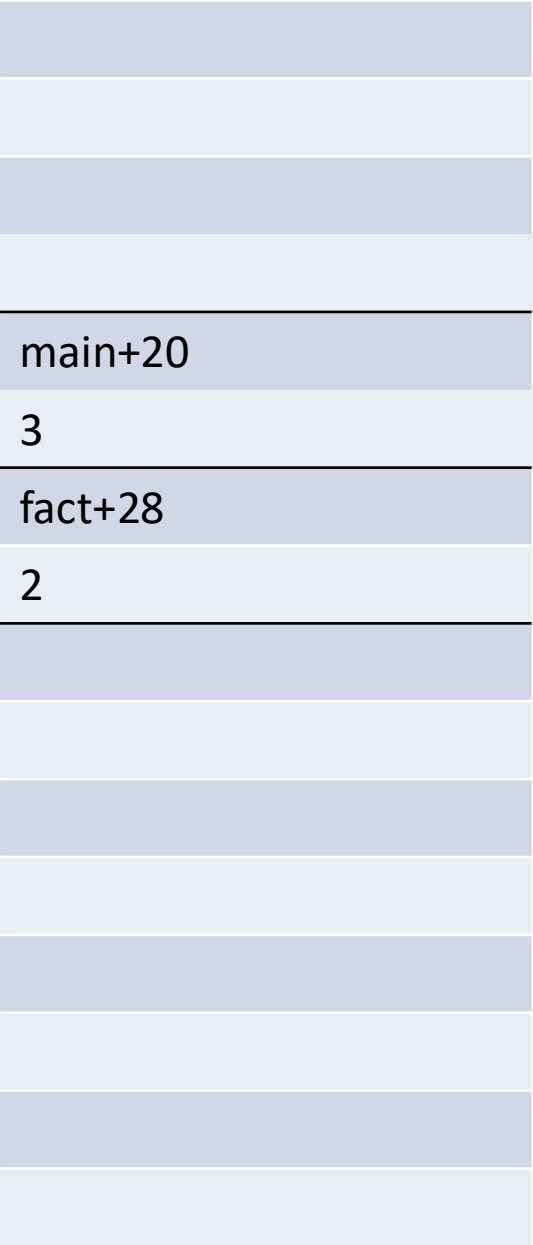
pc →      li      t0, 2
           blt     a0, t0, L1

           addi    a0, a0, -1
           jal     ra, fact
           ld      t0, 0(sp)
           mul     a0, a0, t0

L1:
           ld      ra, 8(sp)
           addi    sp, sp, 16
           jalr    zero, 0(ra)
```

Reg	Value
a0	2
ra	fact+28

sp →



At start of fact(1)

```
# fact(n)
fact:
pc →   addi    sp, sp, -16
        sd      ra, 8(sp)
        sd      a0, 0(sp)

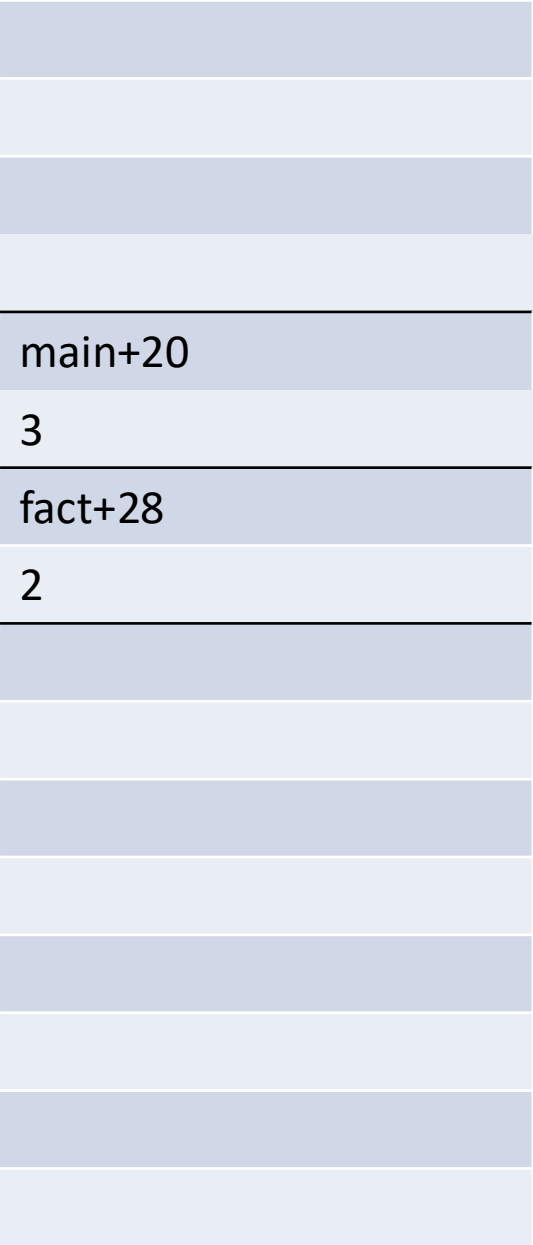
        li      t0, 2
        blt     a0, t0, L1

        addi    a0, a0, -1
        jal     ra, fact
        ld      t0, 0(sp)
        mul     a0, a0, t0

L1:
        ld      ra, 8(sp)
        addi    sp, sp, 16
        jalr    zero, 0(ra)
```

Reg	Value
a0	1
ra	fact+28

sp →



After prologue

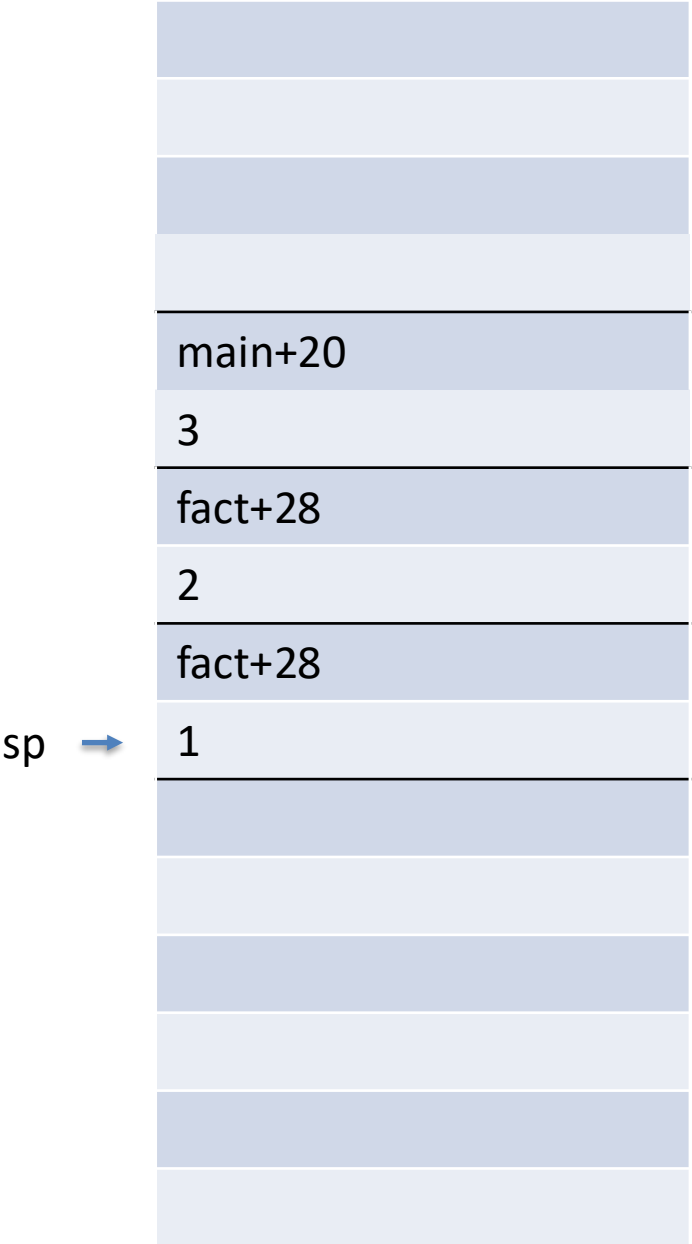
```
# fact(n)
fact:
    addi    sp, sp, -16
    sd      ra, 8(sp)
    sd      a0, 0(sp)

pc →      li      t0, 2
           blt     a0, t0, L1

           addi    a0, a0, -1
           jal     ra, fact
           ld      t0, 0(sp)
           mul     a0, a0, t0

L1:
           ld      ra, 8(sp)
           addi    sp, sp, 16
           jalr    zero, 0(ra)
```

Reg	Value
a0	1
ra	fact+28



Before epilogue

```
# fact(n)
fact:
    addi    sp, sp, -16
    sd      ra, 8(sp)
    sd      a0, 0(sp)

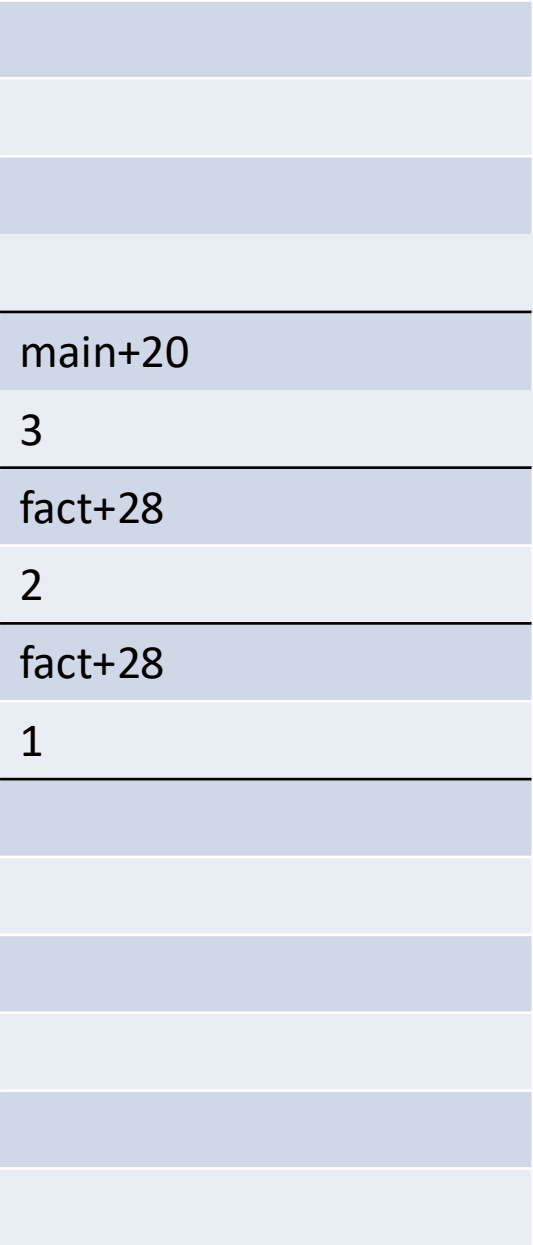
    li      t0, 2
    blt     a0, t0, L1

    addi    a0, a0, -1
    jal     ra, fact
    ld      t0, 0(sp)
    mul     a0, a0, t0

L1:
pc → ld      ra, 8(sp)
    addi    sp, sp, 16
    jalr    zero, 0(ra)
```

Reg	Value
a0	1
ra	fact+28

sp →



After epilogue

fact (n)

fact:

```
addi    sp,  sp, -16
```

sd ra, 8(sp)

```
sd      a0, 0(sp)
```

```
li      t0, 2
```

```
blt    a0, t0, L1
```

```
addi    a0, a0, -1
```

jal ra, fact

ld t0, 0(sp)

```
mul    a0, a0, t0
```

L1:

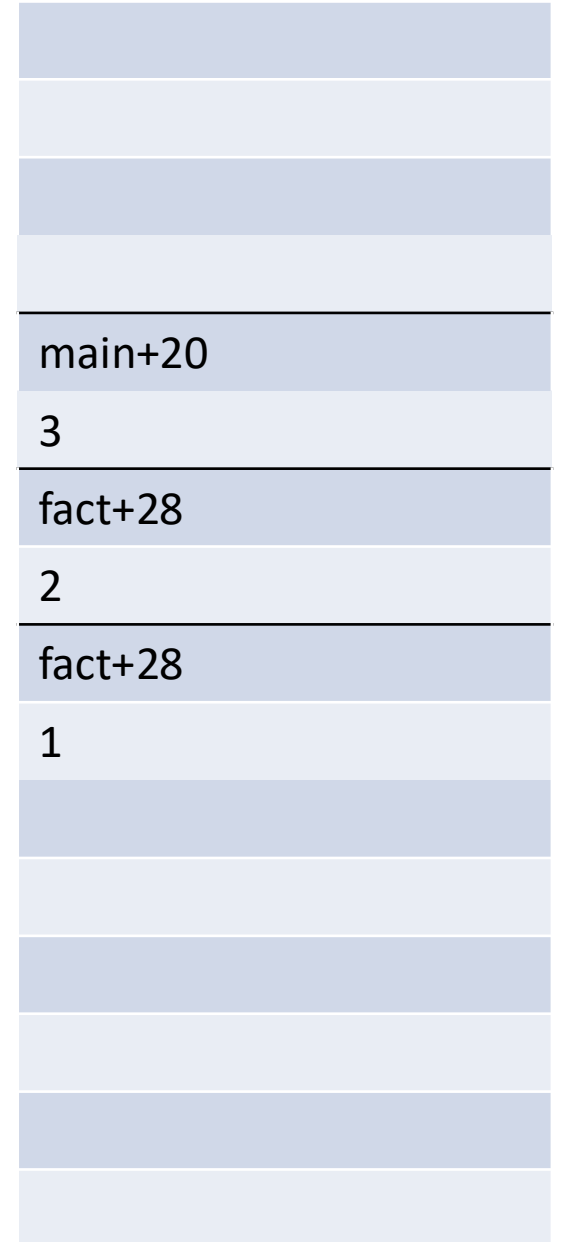
ld ra, 8(sp)

```
addi    sp,  sp, 16
```

pc → jalr zero, 0(ra)

Reg	Value
a0	1
ra	fact+28

sp →



After fact(1)

Reg	Value
a0	1
ra	fact+28

fact (n)

fact:

```
addi    sp, sp, -16
sd      ra, 8(sp)
sd      a0, 0(sp)
```

```
li      t0, 2
blt     a0, t0, L1
```

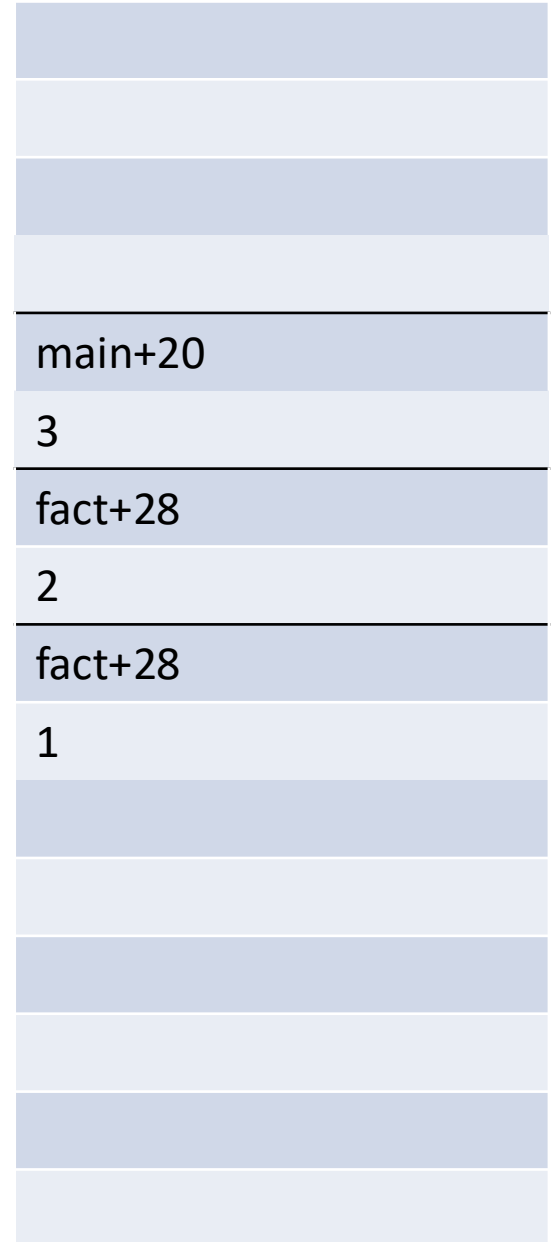
```
addi    a0, a0, -1
jal     ra, fact
ld      t0, 0(sp)
mul     a0, a0, t0
```

pc →

L1:

```
ld      ra, 8(sp)
addi    sp, sp, 16
jalr    zero, 0(ra)
```

sp →



After fact(2)

fact (n)

fact:

```
addi    sp,  sp, -16
sd      ra,  8(sp)
sd      a0,  0(sp)
```

```
li      t0, 2
blt     a0, t0, L1
```

```
addi    a0, a0, -1
jal     ra, fact
ld      t0, 0(sp)
mul     a0, a0, t0
```

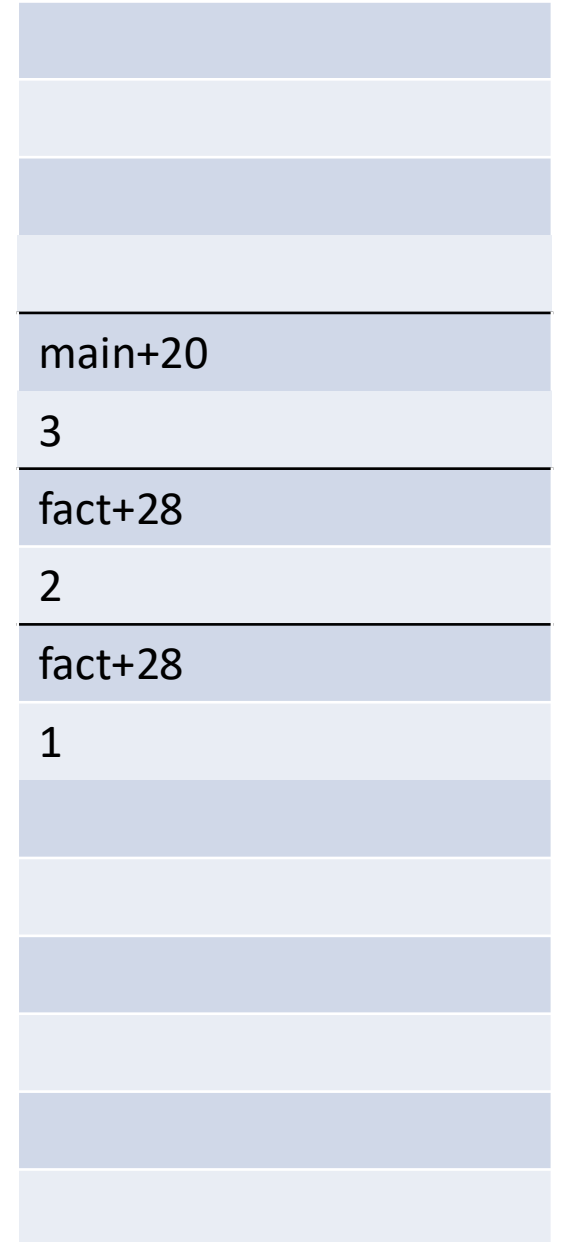
pc →

L1:

```
ld      ra, 8(sp)
addi    sp, sp, 16
jalr    zero, 0(ra)
```

Reg	Value
a0	2
ra	fact+28

sp →



Before returning from fact(3)

fact (n)

fact:

```
addi    sp,  sp, -16
sd      ra,  8(sp)
sd      a0,  0(sp)
```

```
li      t0, 2
blt     a0, t0, L1
```

```
addi    a0, a0, -1
jal     ra, fact
ld      t0, 0(sp)
mul     a0, a0, t0
```

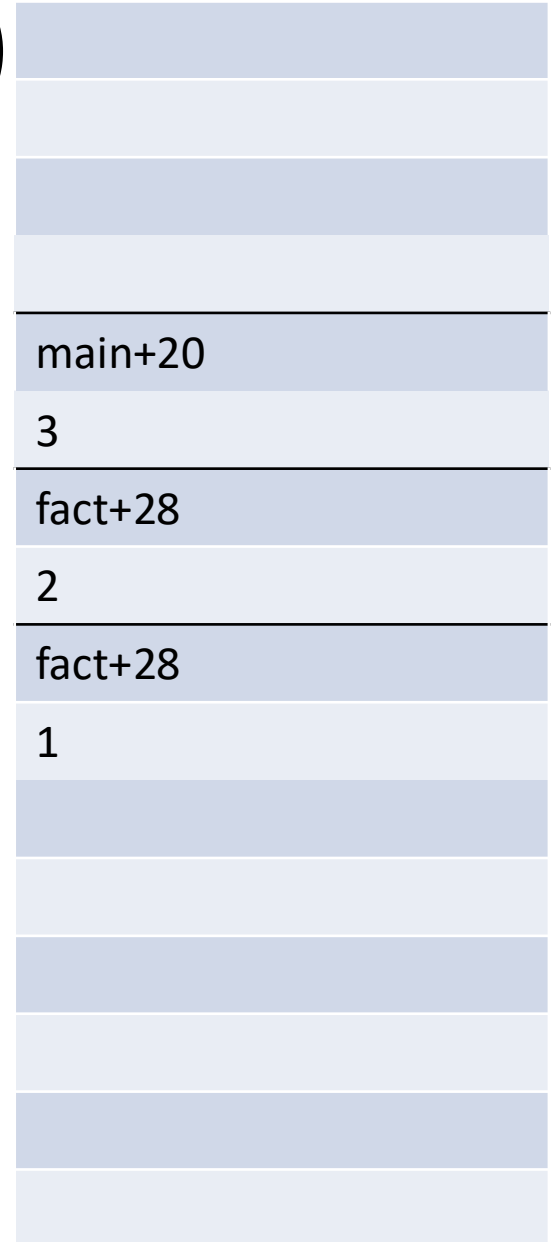
L1:

```
ld      ra, 8(sp)
addi    sp, sp, 16
jalr    zero, 0(ra)
```

pc →

Reg	Value
a0	6
ra	main+20

sp →



Why store registers relative to the stack pointer, rather than at some set memory location?

- A. Saves space.
- B. Easier to figure out where we stored things.
- C. Functions won't overwrite each other's saves.
- D. None of the above

Review: Assembler directives

- Instructions to the assembler
 - `.data` / `.text` / `.rodata` / `.bss` are used to switch between global (mutable) data, executable code, read-only data, and uninitialized data in the output
 - `.dword x` allocates space for 8 bytes with value `x`
 - `.space n` allocates `n` bytes of space
 - `.ascii "string"` writes a 0-terminated string at that location

Review: Arrays!

- How do we declare a 10-doubleword array in our data section?

- Could do

```
.data
```

```
d1:      .dword 0
```

```
d2:      .dword 0
```

```
d3:      .dword 0
```

```
...
```

```
d10:     .dword 0
```

Review: Declaring an Array

- Instead, just declare a big chunk of memory

```
.data
```

```
arr:    .space    80
```

```

.data
arr:    .space 80

.text
    li    t0, 0
    li    t1, 10
    la    s0, arr
loop:
    beq    t0, t1, end
    [What goes here?]
    addi   t0, t0, 1
    j      loop
end:

```

D. More than one of the above

E. None of the above

```

int i;
for (i = 0; i < 10; i++){
    arr[i] = i;
}

```

```
sd    t0, t1(s0)
```

A

```
add    t2, s0, t0
sd     t0, 0(t2)
```

B

```
sd     t0, 0(s0)
addi   s0, s0, 8
```

C

But what if we don't know how big the array will be before runtime?

sbrk system call

- Allocates memory and returns its address in a0
- Amount of memory is specified in bytes in a0
- Used by malloc, new

System Calls

- Syscalls (when we need OS intervention)
 - I/O (print/read stdout/file)
 - Exit (terminate)
 - Get system time
 - Random values

System Calls Review

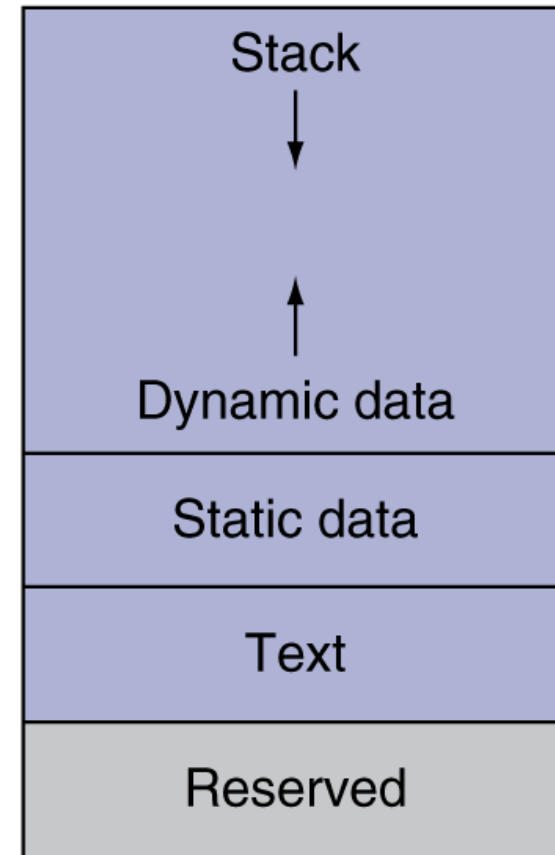
- How to use:
 - Put syscall number into register a7
 - Load arguments into argument registers
 - Issue ecall instruction
 - Retrieve return values

- Example (allocate t4 bytes of memory with sbrk):

```
li      a7, 9      # sbrk system call number
mv      a0, t4     # allocate t4 bytes of mem
ecall
mv      s0, a0     # s0 holds a pointer to mem
```

sbrk allocates memory from which region?

- A. Stack
- B. Dynamic data
- C. Static data
- D. Text
- E. Reserved



What about freeing memory?

- Some operating systems maintain a “program break” which controls the size of the dynamic data
- `sbrk` requests the OS increment/decrement the break
- `malloc()/free()` carve the dynamic data up into chunks which the application can use and maintain lists of free chunks
- Freeing memory adds the chunk to a “free list”
- When more memory is needed, the break is changed

